

CivicFix Internal Docs

AI-verified community crowdfunding & micro-gig marketplace for urban micro-infrastructure repair

Team 5

August 2026

- [1. Problem Statement](#)
 - [The Core Issue — “The Civic Execution Gap”](#)
- [2. The Core Loop](#)
- [3. Tech Stack](#)
 - [Frontend](#)
 - [Backend](#)
 - [Database](#)
 - [AI / Computer Vision](#)
 - [Payments](#)
 - [Maps, Notifications, Infra](#)
- [4. User Roles & Screens](#)
 - [4.1 Citizen — owned by Teammate A](#)
 - [4.2 Organization \(Municipal Admin\) — owned by Teammate B](#)
 - [4.3 Worker \(“Tender”/Contractor\) — owned by Teammate C](#)
 - [4.4 Investor — owned by Teammate D](#)
- [5. Project File Structure](#)
 - [Top Level](#)
 - [Frontend \(React / React Native\)](#)
 - [Backend \(Node.js + Express + MongoDB\)](#)
- [6. AI Integration Points](#)
- [7. Landing Page — Design Notes](#)
- [8. Devpost Submission Content](#)
 - [Built With](#)
 - [Project Story](#)

1. Problem Statement

Project Title: CivicFix **Tagline:** Decentralizing urban repair through AI-verified community crowdfunding and micro-gig economies.

The Core Issue — “The Civic Execution Gap”

Across urban and semi-urban landscapes, local communities suffer from deteriorating micro-infrastructure — dangerous potholes, broken streetlights, unkempt public parks, and damaged sidewalks. While modern cities provide digital portals for citizens to report these issues, these systems suffer from the **Civic Execution Gap**: they successfully digitize the complaint, but they completely fail to digitize the solution.

The Three Pillars of Systemic Failure

1. The Municipal Bottleneck (Resource Deficit) Local governments are plagued by bureaucratic routing, budget constraints, and severe bandwidth limits. Minor infrastructure defects, while highly disruptive to daily neighborhood life, are placed at the bottom of municipal priority lists. A simple \$50 pothole repair can take six months to process, approve, and execute.

- 2. The Black-Box Reporting Experience (Citizen Apathy)** Current “civic tech” apps function as black boxes. A citizen snaps a photo, submits it, and rarely receives a follow-up. This one-way communication model breeds civic apathy; residents stop reporting issues because they feel powerless and disconnected from the actual improvement of their local environment.
- 3. The Untapped Hyper-Local Workforce (Economic Inefficiency)** Simultaneously, local gig workers, independent contractors, and handymen are actively seeking small, immediate jobs. However, there is no centralized, trustless marketplace connecting a neighborhood’s immediate willingness to improve its surroundings with the local workforce capable of executing the job.

The Ramifications

This broken loop leads to prolonged safety hazards, decreased neighborhood morale, and a missed opportunity to foster local economic growth. The traditional top-down approach to urban maintenance is fundamentally unscalable.

The Opportunity (“Why Now?”)

Until now, allowing citizens to crowdfund and hire out public repairs was impossible due to a lack of trust and verification. How do donors know the job was actually done? With the advent of accessible multimodal AI, we can now instantly estimate repair costs via computer vision, pool micro-donations securely, and use AI image analysis to mathematically verify the completed work before releasing funds.

It is time to shift from municipal dependency to decentralized, AI-empowered community action.

2. The Core Loop

1. **Citizen reports an issue** — snaps a photo of a pothole/streetlight/etc., app auto-tags location.
2. **AI estimates repair cost** — computer vision looks at the photo and estimates severity + cost.
3. **Campaign auto-created** — a micro-crowdfunding campaign is generated for that exact repair.
4. **Neighbors fund it** — small donations pool until the target is hit.
5. **Worker claims the job** — a local handyman/contractor accepts it from the job marketplace.
6. **Worker completes & submits proof** — uploads an “after” photo.
7. **AI verifies completion** — compares before/after photos to confirm the job matches what was paid for.
8. **Funds released** — escrow releases payment to the worker only after AI verification passes.

3. Tech Stack

Frontend

Component	Tech	Why
Mobile app (citizens + workers)	React Native	One codebase for Android/iOS
Web dashboard (admin/municipal)	React (Next.js) + Tailwind CSS	Fast to build, polished control-panel feel

Backend

Component	Tech	Why
API server	Node.js + Express (REST API)	Consistent JS across stack
Auth	Firebase Auth / JWT	Avoid building auth from scratch
Real-time updates	WebSockets / Firebase Realtime DB	Live status changes

Database

Component	Tech	Why
Primary DB	MongoDB (Mongoose)	Flexible schema for users, campaigns, jobs, transactions
Image storage	AWS S3 / Cloudinary	Before/after repair photos

AI / Computer Vision

Component	Tech	Why
Cost estimation & completion verification	Multimodal AI API (e.g. Google Gemini free tier)	Fast to integrate, handles image + text reasoning in one call

Payments

Component	Tech	Why
Payment collection	Razorpay	India-first, supports UPI
Escrow logic	Custom (DB-tracked pending state)	Funds hold until AI verification passes

Maps, Notifications, Infra

Component	Tech
Maps	Google Maps API / Mapbox
Notifications	Firebase Cloud Messaging + Twilio (SMS)
Hosting	Vercel (frontend) + Railway/Render (backend)
CI/CD	GitHub Actions

4. User Roles & Screens

The product has **4 user roles**, each with its own app experience built on a shared UI shell (feed of cards + detail thread + per-role messages inbox + profile grid), Instagram/Reddit-inspired.

4.1 Citizen — owned by Teammate A

#	Screen	Purpose
1	HomeFeedScreen	Feed of nearby issues/campaigns
2	CommunityFeedScreen	Explore/discover, filterable by category
3	ReportIssueScreen	Camera capture + geo-tag + category picker
4	CostEstimateScreen	AI cost estimate preview before posting
5	IssueDetailScreen	Photo, funding bar, status timeline, comments
6	DonateScreen	Fund another citizen’s reported issue
7	MessagesScreen	DM-style threads about your own reports
8	NotificationsScreen	Funding milestones, job claimed, work completed
9	ProfileScreen	Grid of your own reported issues
10	LeaderboardScreen	Civic points, badges

Demo-critical: ReportIssueScreen → CostEstimateScreen → DonateScreen.

4.2 Organization (Municipal Admin) — owned by Teammate B

#	Screen	Purpose
1	OrgDashboardScreen	Overview: pending reports, active jobs, funded, completion rate
2	ReportsQueueScreen	New citizen reports awaiting review
3	VerifyReportScreen	Approve/reject with AI feature detection
4	MarketplacePushScreen	Approved issue becomes an open job
5	ActiveJobsScreen	Kanban: Open → Claimed → In Progress → Submitted → Verified
6	DisputeScreen	Handle flagged jobs
7	WorkerDirectoryScreen	Registered workers, ratings, verification status
8	AnalyticsScreen	Heatmap of issue density, response time, category breakdown
9	MessagesScreen	DM threads with citizens/workers
10	TeamSettingsScreen	Add org staff, permission levels
11	SettingsScreen	Org profile, notification & escrow preferences

Demo entry: “One-Click Demo Login — Organization” (role organization, reg id MC-MUM-2026-99).

4.3 Worker (“Tender”/Contractor) — owned by Teammate C

#	Screen	Purpose
1	WorkerOnboardingScreen	Upload ID, select skill category + license
2	JobFeedScreen	Nearby open jobs, sortable by distance/payout/urgency
3	JobDetailScreen	Issue photos, location, AI cost estimate, funding status
4	SubmitBidScreen	Quote price + timeline (tender comment)
5	BidStatusScreen	Pending / Awarded / Rejected
6	ActiveJobScreen	Navigation to site, checklist, instructions
7	UploadProofScreen	Mandatory “after” photo capture
8	VerificationStatusScreen	AI pass/fail/pending before-after match
9	MessagesScreen	DM threads with org/citizen about specific jobs
10	EarningsScreen	Balance, withdrawal, payment history
11	ReviewsScreen	Citizen/org feedback on completed jobs
12	WorkerProfileScreen	Portfolio grid of before/after job photos, badges

Demo entry: “One-Click Demo Login — Worker” — flows through marketplace → bid → active job → proof upload → AI verification → wallet → profile.

4.4 Investor — owned by Teammate D

#	Screen	Purpose
1	InvestorDashboardScreen	Campaigns needing funding, sortable Hot/Top/New
2	CampaignDetailScreen	Cost estimate, worker bids, engagement, verification status
3	TrustScoreScreen	AI confidence + org track record + worker rating
4	FundingDecisionScreen	Set contribution amount / accept a worker’s quote

5	PortfolioScreen	All funded projects, live status
6	CompletionReportScreen	Before/after AI comparison, impact summary
7	PayoutConfirmScreen	Confirms funds released to worker
8	MessagesScreen	DM threads with orgs/workers on funded projects
9	RegionalAnalyticsScreen	Jobs completed, \$ deployed, areas improved
10	InvestorSettingsScreen	Payment methods, KYC, notification preferences

Demo-critical: InvestorDashboardScreen → FundingDecisionScreen → PayoutConfirmScreen.

5. Project File Structure

Structure is split so each teammate works inside their own role folder on both frontend and backend — minimizing merge conflicts.

Top Level

```
civicfix/
├─ frontend/
├─ backend/
├─ shared/           # types/constants shared by both – PR review required
├─ .gitignore
├─ README.md
└─ docker-compose.yml
```

Frontend (React / React Native)

```
frontend/src/
├─ screens/
│  └─ shared/           # Login, Signup, RoleSelect, Onboarding, Splash
│  └─ citizen/          # Teammate A owns this folder only
│  └─ organization/     # Teammate B owns this folder only
│  └─ worker/           # Teammate C owns this folder only
│  └─ investor/         # Teammate D owns this folder only
├─ components/
│  └─ common/           # shared, PR review required
│  └─ citizen/ organization/ worker/ investor/
├─ navigation/
│  └─ RootNavigator.jsx (shared)
│  └─ CitizenNavigator.jsx / OrgNavigator.jsx / WorkerNavigator.jsx / InvestorNavigator.jsx
├─ api/
│  └─ citizenApi.js / organizationApi.js / workerApi.js / investorApi.js / authApi.js
├─ context/             # global state – shared, edit carefully
├─ theme/colors.js      # design tokens – green=citizen, blue=org, orange=worker, purple=investor
└─ App.jsx
```

Backend (Node.js + Express + MongoDB)

```
backend/src/
├─ models/              # Mongoose schemas – shared, mostly additive
│  └─ User.js (role field: citizen/org/worker/investor)
│  └─ Issue.js / Campaign.js / Job.js / Bid.js / Transaction.js / Review.js
├─ routes/
│  └─ auth.routes.js (shared)
│  └─ citizen.routes.js / organization.routes.js / worker.routes.js / investor.routes.js
├─ controllers/
│  └─ citizen.controller.js / organization.controller.js / worker.controller.js / investor.controller.js
├─ services/
│  └─ ai.service.js      # cost estimate + verification calls
│  └─ payment.service.js # Razorpay/escrow logic
│  └─ notification.service.js
│  └─ upload.service.js
```

```
├─ middleware/
│   └─ auth.middleware.js / role.middleware.js / errorHandler.js
└─ config/db.js
```

Why this avoids merge conflicts: each role has its own screen folder, component folder, API file, route file, and controller file. Only shared files (RootNavigator, models/, theme/colors.js, routes/index.js) require coordinated, reviewed PRs.

6. AI Integration Points

CivicFix needs **one reusable multimodal AI service** (image + text), called from four different features with different prompts — not four separate integrations.

#	Feature	Screen(s)	Needs live AI call?	Suggested free API
1	Cost estimation	CostEstimateScreen	Yes	Google Gemini API (free tier)
2	Completion verification	VerificationStatusScreen	Yes	Google Gemini API (free tier)
3	Report pre-screening	VerifyReportScreen	Yes (light use)	Google Gemini API (free tier)
4	Trust score	TrustScoreScreen	No — computed formula	—
5	Dispute re-check	DisputeScreen	Yes	Google Gemini API (free tier)
6	Comment moderation (optional)	any comment thread	Optional	Hugging Face Inference API
7	Feed ranking (optional)	Community/Investor feeds	No — computed formula	—

Getting a free key: aistudio.google.com/app/apikey → sign in → “Create API key.” Store as `AI_API_KEY` in the backend `.env`, never in frontend code.

Shared service file:

```
backend/src/services/ai.service.js
├─ estimateCost(photoUrl, category)
├─ verifyCompletion(beforeUrl, afterUrl)
├─ prescreenReport(photoUrl, category)
└─ reviewDispute(beforeUrl, afterUrl, note)
```

7. Landing Page — Design Notes

Direction: dark, animated, community-energy — Discord-inspired shell with its own palette, not Discord’s colors.

- **Palette:** Asphalt Night #0E1319 (background), Safety Orange #FF6A3D (CTA/energy), Verified Teal #00D9A3 (AI-trust accent), Warm Sand #F4C77B (accent), off-white text #F3F0E9.
- **Type:** Space Grotesk (headlines), Inter (body), JetBrains Mono (stats).
- **Signature element:** an interactive before/after drag slider in the hero demonstrating the AI verification mechanic directly.
- **Sections:** Nav → Hero (with slider) → Live ticker of “just fixed” issues → Problem stats → How it works (4-step loop) → Role cards (4, bento grid) → AI verification feature split → Testimonial marquee (one voice per role) → Final CTA band → Footer.
- **Note:** shipped demo build uses original SVG illustrations rather than stock photography; production build should swap in licensed Indian street/community photography.

8. Devpost Submission Content

Built With

react · react-native · javascript · html5 · css3 · node.js · express · rest-api · mongodb · mongoose · google-gemini · computer-vision · multimodal-ai · generative-ai · jwt · firebase-auth · razorpay · google-maps-api · clouinary · github · github-actions · figma · vercel · postman · twilio · firebase-cloud-messaging

Project Story

Inspiration

Every one of us has walked past the same pothole for weeks, watched a streetlight stay dark for months, or reported an issue on a municipal app only to get a ticket number and silence. Civic tech has spent a decade digitizing the *complaint* — but nobody digitized the *fix*. Local governments are bottlenecked by budget and bureaucracy, small repairs are too tiny to prioritize and too disruptive to ignore, and meanwhile there's a whole hyper-local workforce of contractors and handymen sitting idle, ready to do the job for a fair price. We kept coming back to one question: what if a neighborhood could just fix its own street? Not by replacing local government, but by giving citizens, verified workers, and civic-minded funders a trustworthy, AI-backed way to close the loop themselves. That question became CivicFix.

What it does

CivicFix turns a broken streetlight or a pothole into a fundable, trackable, verifiable repair project — no municipal form required. A citizen photographs the issue and AI instantly estimates severity and repair cost, turning the report into a live micro-crowdfunding campaign. The local organization reviews and verifies it, then pushes it into an open job marketplace. A local worker browses nearby jobs, submits a quote, and completes the repair. A civic-minded investor can back verified, high-trust campaigns and see exactly where their money goes. Once the worker uploads an “after” photo, AI compares it against the original “before” photo to confirm the job actually matches what was paid for — only then does escrowed money release to the worker.

How we built it

We split the product into four role-based experiences — Citizen, Organization, Worker, and Investor — each with its own screen flow, feed-style UI, and dedicated messaging inbox, but all hitting one shared backend and one shared AI service. Frontend: React / React Native with a dark, animated UI and a Reddit-style feed pattern. Backend: Node.js + Express REST API with MongoDB. AI layer: a single multimodal image+text service reused across four prompts (cost estimation, completion verification, report pre-screening, dispute review). We organized the codebase so each teammate owned their own screen folder, API file, controller, and route file end to end, letting four of us build in parallel with almost zero merge conflicts. The landing page centers on an interactive before/after slider so visitors can feel the AI verification mechanic directly.

Challenges we ran into

Designing four apps that feel like one product was the first big challenge — we standardized on one shared UI shell (feed + detail + messages + profile) across all four roles to keep things cohesive. Making AI verification actually reliable took iteration: naive before/after photo comparison was thrown off by angle and lighting, so we had to prompt the model with explicit context about the issue type and location. Avoiding merge conflicts across a 4-person team on a tight clock meant planning the file structure and ownership boundaries before writing feature code. And keeping the trust loop honest meant making sure the “AI verified” badge is a real gate in the escrow logic, not just a UI label.

Accomplishments that we're proud of

A working end-to-end loop — report, AI cost estimate, crowdfund, worker claims job, AI-verified

completion, payout — across four distinct, fully-built role experiences. A single, reusable AI service powering four different features instead of four separate integrations. A codebase structured well enough that four teammates could build simultaneously without stepping on each other's files. A landing page that lets a visitor try the core trust mechanic themselves.

What we learned

How much of “trust” in a decentralized system comes down to one well-designed verification gate, not a dozen scattered safety features. That good multi-role UX means finding the one shared pattern that lets very different user journeys still feel like the same app. That planning file structure and ownership boundaries before the clock starts is one of the highest-leverage things a small team can do. And that multimodal AI is genuinely good at “does this photo show a fixed pothole” with the right prompt — but needs testing against messy, real-world photo pairs.

What's next for CivicFix

Pilot the platform in a real neighborhood with real municipal partnership, starting with one ward and one issue category. Add dispute resolution tooling so flagged jobs get a fast, fair second look from both AI and a human moderator. Build out worker verification (ID, licensing, ratings) so trust flows both ways. Explore partnerships with municipal bodies to formally recognize AI-verified community repairs.